Упражнение 5 Изглед на камерата. Проекции.

Целта на упражнението е да конфигурира изгледа на камерата в сцена, рендирана с библиотеката LWJGL, както и да се разберат и приложат различните видове проекции в компютърната графика.

Основни моменти:

- Да се разберат принципите на преобразуване на сцената спрямо изгледа на камерата.
- Да се изяснят понятията перспективна и ортографска проекция.
- Реализация на посочените трансформации в графична среда.

След упражнението студентите ще могат да конфигурират и манипулират изгледа на сцена чрез проекции и ще придобият практически умения за работа с JOML и LWJGL при настройване на камерата.

Припомняме, че етапът на обработка на върховете в програмируемият графичен пайплайн включва следните основни моменти:

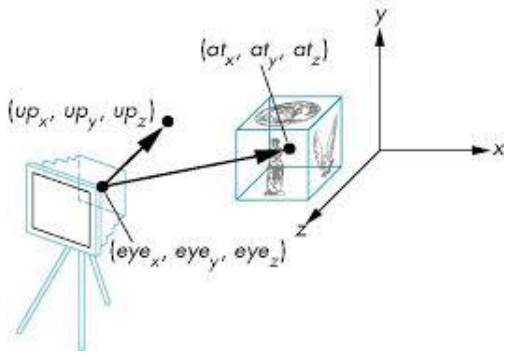
✓ Моделна трансформация - преобразува координатите на обектите от локалното пространство (Object Space) към световното пространство (World Space). Тук се прилагат трансформации като мащабиране, ротация и транслация.

✓ Камера (изглед, гледна точка) - преобразува сцената спрямо изгледа на камерата (от World Space към View Space). Позицията и ориентацията на камерата се прилагат, като се използва матрица на изгледа.

✓ Проекция - тук 3D координатите се преобразуват в clip space (пространството на отсичане). Този етап определя дали се използва перспективна или ортографска проекция. Прилага се матрица на проекцията, която деформира сцената така, че да отговаря на перспективните или ортографските правила.

1. **Позиция на камерата**

Тази трансформация конвертира координатите от световната координатна система в координатната система на камерата. Камерата обикновено е разположена на определена позиция с дадена ориентация и трябва да трансформира цялата сцена така, че тя да изглежда спрямо нейния изглед.



Три момента трябва да бъдат дефинирани по отношение на позицията на камерата:

- eye – позицията на камерата.
- at – точката, към която гледа камерата.
- up – дефинира ориентацията на камерата.

Камерата не се движи физически, а цялата сцена се трансформира спрямо нея. Това се прави чрез view матрица (матрица на изгледа), която премества и завърта света така, сякаш камерата е на (0,0,0) и гледа по оста -z (в OpenGL стандартно). Така ако искаме да преместим камерата напред, технически местим цялото световно пространство назад.

За да се дефинира координатната система на камерата, се изчисляват три взаимно перпендикулярни вектора:

- Вектор напред (forward) - сочи от камерата към целевата точка

$$\text{forward} = \frac{\text{target} - \text{eye}}{|\text{target} - \text{eye}|}$$

- Вектор надясно (right) - определя "дясната" страна на камерата

$$\text{right} = \frac{\text{forward} \times \text{up}}{|\text{forward} \times \text{up}|}$$

- Вектор нагоре (up) – преизчислява се, за да се гарантира ортогоналност спрямо оста z.

$$\mathbf{up} = \mathbf{forward} \times \mathbf{right}$$

Матрицата на изгледа се използва за трансформиране на обектите от световното пространство в пространството на камерата.

$$V = \begin{bmatrix} r_x & r_y & r_z & -\text{dot}(r, eye) \\ u_x & u_y & u_z & -\text{dot}(u, eye) \\ -f_x & -f_y & -f_z & -\text{dot}(f, eye) \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

където:

right = (r_x, r_y, r_z)

up = (u_x, u_y, u_z)

forward = (f_x, f_y, f_z)

2. Перспективна и ортографска проекция

Проекцията е важен етап в графичния рендеринг и е отговорна за преобразуването на 3D координатите на сцената в 2D координати за екранното представяне. Проекцията е необходима, защото компютърните екрани са двуизмерни (2D), а обектите в сцената често са триизмерни (3D).

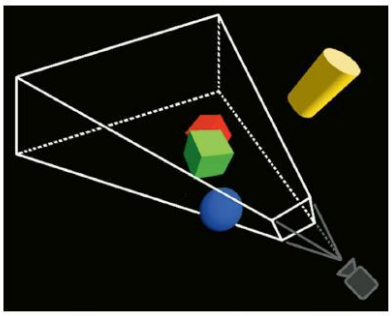
В реалния свят ние възприемаме околната среда чрез перспектива – обекти, които са по-далеч, изглеждат по-малки. В компютърната графика трябва изрично да се дефинира начинът, по който 3D обектите ще бъдат изобразени на 2D екран. Това се постига чрез проекционна трансформация.

Проекцията може да бъде:

а) Перспективна проекция - имитира човешкото зрение: далечните обекти изглеждат по-малки, а близките – по-големи. Създава усещане за дълбочина.

б) Ортографска проекция – не прилага перспективно скалиране. Обектите изглеждат с еднакъв размер, независимо от разстоянието.

За да изясним ролята на проекцията и начините на нейното приложение, ще се върнем на понятие, което дефинирахме в лекция 2 – фрустум (пространството, видимо от камерата).

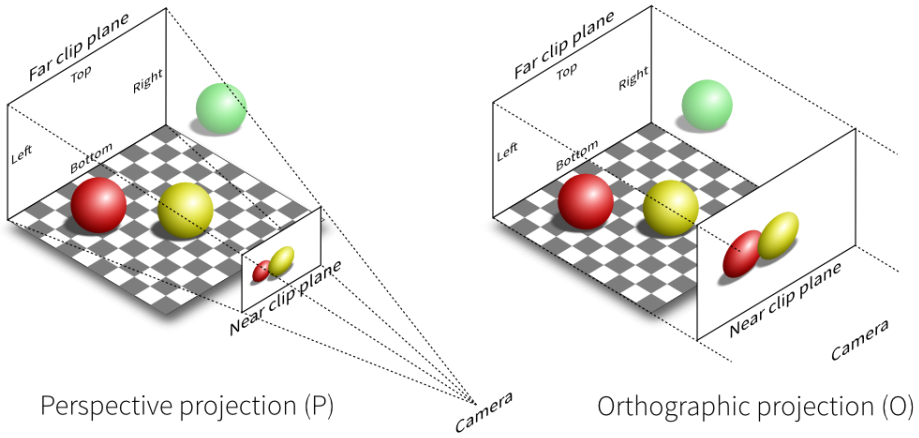


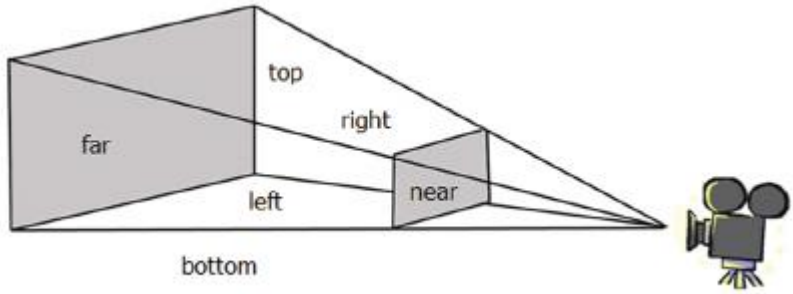
Фрустумът е пирамидален обем, който определя кои обекти са видими в сцената и кои трябва да бъдат отрязани. Той определя границите на рендериране (near/far planes); подобрява производителността, като отрязва невидими обекти (clipping); определя изкривяването на перспективата.

Определя се от 6 равнини:

- Near plane – близката равнина на видимост. Определя най-близката точка, която може да бъде видяна.
- Far plane – далечната равнина на видимост. Задава най-далечната точка, която се рендерира
- Top plane – горна граница.
- Bottom plane – долна граница.
- Left plane – лява граница.

- Right plane – дясна граница.

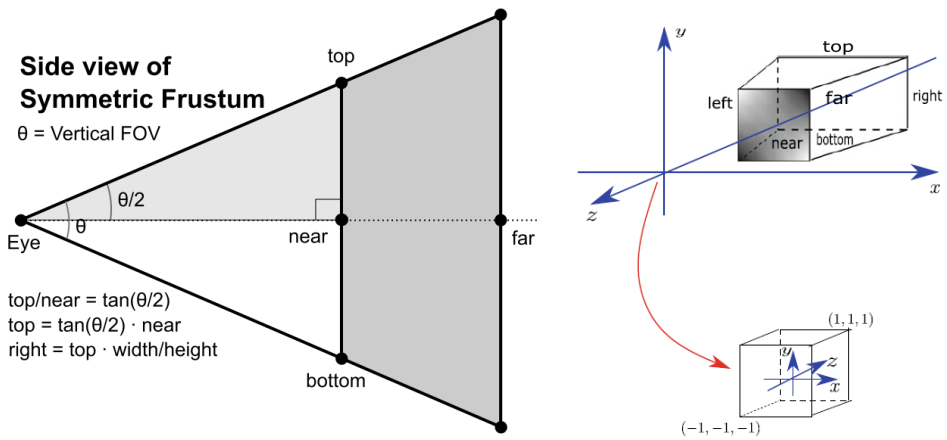




Обектите извън този обем не се рендерират.

При **перспективната проекция** фрустумът има пресечена пирамидална форма. Определя се чрез следните ключови параметри:

- Field of View (FOV) – ъгълът на видимост.
- Aspect Ratio – съотношение на ширина към височина.
- Near и Far Clipping Planes – ограничават видимите обекти.



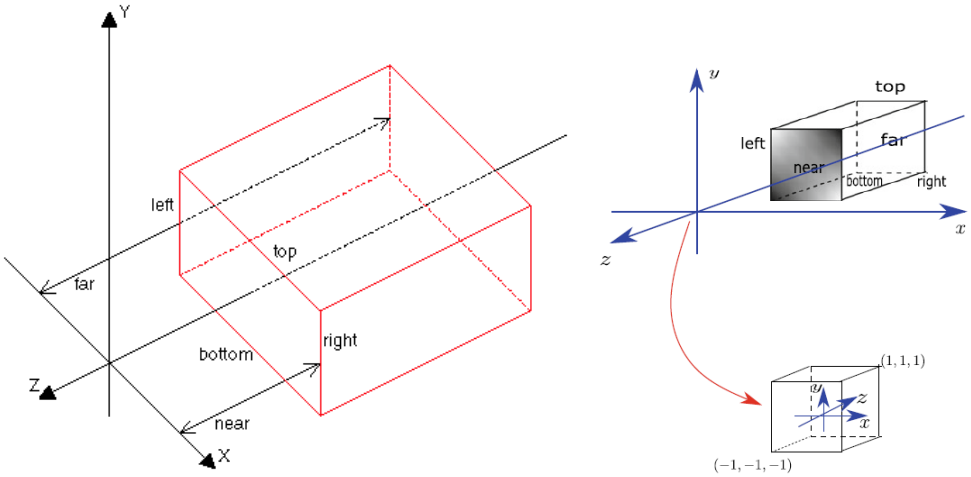
Матрицата за перспективна проекция е:

$$P = \begin{bmatrix} \frac{1}{aspect \cdot \tan(FOV/2)} & 0 & 0 & 0 \\ 0 & \frac{1}{\tan(FOV/2)} & 0 & 0 \\ 0 & 0 & \frac{far+near}{near-far} & \frac{2 \cdot far \cdot near}{near-far} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

Матрицата на перспективната проекция мащабира x- и y-координатите в зависимост от ъгъла на виждане и съотношението между страните (aspect ratio). Освен това тя преобразува z-координатата така, че дълбочината да бъде коректно представена спрямо близката и далечната равнина. След прилагане на проекционната матрица се извършва деление по хомогенната координата w (известно като перспективно делене), което води до нормализация на координатите в каноничното пространство (Normalized Device Coordinates, NDC), където всяка координата попада в интервала $[-1,1]$. В резултат на това по-далечните обекти изглеждат по-малки.

$$x_{ndc} = \frac{x_{clip}}{w_{clip}}, \quad y_{ndc} = \frac{y_{clip}}{w_{clip}}, \quad z_{ndc} = \frac{z_{clip}}{w_{clip}}$$

При **ортографската проекция** фрустумът има форма на правоъгълен паралелепипед, за разлика от пресечената пирамида при перспективната проекция. Near и Far Plane имат еднаква ширина и височина, а страните на фрустума са успоредни. Обектите не променят размера си в зависимост от разстоянието, като всички линии, които са паралелни в 3D, остават паралелни в 2D.



Ортографската проекция преобразува 3D координатите в 2D координати, като линейно мащабира координатите, без да прилага перспективно деление.

Ако зададем следния обем за рендиране:

- граници по X - left (l), right (r);
- граници по Y - bottom (b), top (t);
- граници по Z - near (n), far (f),

ортографската проекционна матрица ще има следния вид:

$$P = \begin{bmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{r+l}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & -\frac{2}{f-n} & -\frac{f+n}{f-n} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

При ортографската проекция нормализацията до каноничното пространство се извършва директно чрез проекционната матрица, без да възниква перспективно деление, тъй като хомогенната координата w остава равна на 1.

В допълнение:

След получаване на нормализираните координати (NDC), те се преобразуват в координати на устройството чрез т.нар. viewport трансформация. При нея интервалът $[-1,1]$ се преобразува в интервала [0, width] за хоризонталната координата и [0, height] за вертикалната координата. Това се описва със следните зависимости:

$$x_{screen} = \frac{x_{ndc}+1}{2} \cdot width, \quad y_{screen} = \frac{y_{ndc}+1}{2} \cdot height, \quad z_{screen} = \frac{z_{ndc}+1}{2}$$

3. Реализация на камера и перспективи в графична среда

Като основа ще използваме проекта от предходния учебен час (наличен е в бранч Lab5). За да можем да проследим ролята на камерата и реализираната проекция, е нужен реален 3D обект. Нека да създадем куб. За Ваше улеснение предоставяме координатите му. Координатите са разписани за примитив GL_TRIANGLES, ако желаете да използвате друг, е нужно да модифицирате данните.

```
//Предна стена
-0.5f, -0.5f,  0.5f, 1f, 0f, 0f,
0.5f, -0.5f,  0.5f, 1f, 0f, 0f,
0.5f,  0.5f,  0.5f,  1f, 0f, 0f,
0.5f,  0.5f,  0.5f,  1f, 0f, 0f,
-0.5f,  0.5f,  0.5f,  1f, 0f, 0f,
-0.5f, -0.5f,  0.5f,  1f, 0f, 0f,

// Задна стена
-0.5f, -0.5f, -0.5f,  1f, 1f, 0f,
0.5f, -0.5f, -0.5f, 1f, 1f, 0f,
0.5f,  0.5f, -0.5f,  1f, 1f, 0f,
0.5f,  0.5f, -0.5f,  1f, 1f, 0f,
-0.5f,  0.5f, -0.5f, 1f, 1f, 0f,
-0.5f, -0.5f, -0.5f,  1f, 1f, 0f,

// Лява стена
-0.5f,  0.5f,  0.5f,  1f, 1f, 0f,
-0.5f,  0.5f, -0.5f,  1f, 1f, 0f,
-0.5f, -0.5f, -0.5f, 1f, 1f, 0f,
```

```
        -0.5f, -0.5f, -0.5f, 1f, 1f, 0f,
        -0.5f, -0.5f,  0.5f,  1f, 1f, 0f,
        -0.5f,  0.5f,  0.5f, 1f, 1f, 0f,

        // Дясна стена
        0.5f,  0.5f,  0.5f, 1f, 0f, 1f,
        0.5f,  0.5f, -0.5f,  1f, 0f, 1f,
        0.5f, -0.5f, -0.5f,  1f, 0f, 1f,
        0.5f, -0.5f, -0.5f,  1f, 0f, 1f,
        0.5f, -0.5f,  0.5f,  1f, 0f, 1f,
        0.5f,  0.5f,  0.5f,  1f, 0f, 1f,

        // Горна страна
        -0.5f,  0.5f, -0.5f, 0f, 1f, 1f,
        0.5f,  0.5f, -0.5f, 0f, 1f, 1f,
        0.5f,  0.5f,  0.5f,  0f, 1f, 1f,
        0.5f,  0.5f,  0.5f,  0f, 1f, 1f,
        -0.5f,  0.5f,  0.5f,  0f, 1f, 1f,
        -0.5f,  0.5f, -0.5f,  0f, 1f, 1f,

        // Долна страна
        -0.5f, -0.5f, -0.5f,  0f, 1f, 0f,
        0.5f, -0.5f, -0.5f,  0f, 1f, 0f,
        0.5f, -0.5f,  0.5f, 0f, 1f, 0f,
        0.5f, -0.5f,  0.5f, 0f, 1f, 0f,
        -0.5f, -0.5f,  0.5f, 0f, 1f, 0f,
        -0.5f, -0.5f, -0.5f,  0f, 1f, 0f
```

За да можете да видите куба от всички негови посоки, задайте му непрекъснато въртеливо движение спрямо трите оси на координатната система. Как се реализира това разгледахме в предходното упражнение. Подсказка: модифицирайте метода update() в класа на куба.

Да реализираме логиката, свързана с добавянето на камера и проекция. Създайте нов пакет **camera**. В него добавете абстрактен клас **Camera**. В него ще се имплементира основната логика, свързана със задаването на трите вектора, описващи камерата – позиция (полето position); точка, към която гледа (target) и ориентация (up). Класът съдържа двете ключови матрици: матрица на наблюдение (viewMatrix) и матрица на проекцията (projectionMatrix). За извличането на изчислената матрица на камерата е отговорен методът getViewMatrix(), докато матрицата на проекцията е реализирана в наследниците, в зависимост от избраният вариант на проекция (перспективна или ортографска).

```
package bg.tu_varna.sit.gs.camera;

import org.joml.Matrix4f;
import org.joml.Vector3f;
```

```
public abstract class Camera {
   private Vector3f position = new Vector3f(0, 0, 3);
   private Vector3f target = new Vector3f(0, 0, 0);
   private Vector3f up = new Vector3f(0, 1, 0);

   private final Matrix4f viewMatrix = new Matrix4f();
   private final Matrix4f projectionMatrix = new Matrix4f();

   public void setPosition(Vector3f position) {
      this.position = position;
   }

   public void setTarget(Vector3f target) {
      this.target =  target;
   }

   public void setUp(Vector3f up) {
      this.up = up;
   }


   public Matrix4f getViewMatrix() {
      viewMatrix.identity().lookAt(position, target, up);
      return viewMatrix;
   }

   public Matrix4f getProjectionMatrix() {
      return projectionMatrix;
   }

   public abstract void updateProjection();
}
```

Създайте клас PerspectiveCamera, разширяващ Camera. Той ще поеме отговорността, в случай че изберем да приложим перспективна проекция. В него ще зададем основните характеристики на този вид проекция: ъгъл на видимост, съотношение между страните, отстояние до near и far plane.

```
package bg.tu_varna.sit.gs.camera;

public class PerspectiveCamera extends Camera {

   private final float fov, near, far, aspectRatio;

   public PerspectiveCamera(float fov, float near, float far, int width, int height) {
      this.fov = fov;
```

```
        this.near = near;
        this.far = far;

        this.aspectRatio = (float) width / height;

        updateProjection();
    }

    @Override
    public void updateProjection() {
        getProjectionMatrix().identity().perspective(fov, aspectRatio, near, far);
    }
}
```

Създайте клас OrthographicCamera, разширяващ Camera. Той ще има грижата да имплементира ортографска проекция. Основните параметри на този вид проекция описват шестте гранични равнини на фрустума.

```
package bg.tu_varna.sit.gs.camera;

public class OrthographicCamera extends Camera {
    private final float left, right, bottom, top, near, far;

    public OrthographicCamera(float left, float right, float bottom, float top, float near, float far) {
        this.left = left;
        this.right = right;
        this.bottom = bottom;
        this.top = top;
        this.near = near;
        this.far = far;

        updateProjection();
    }

    @Override
    public void updateProjection() {
        getProjectionMatrix().identity().ortho(left, right, bottom, top, near, far);
    }
}
```

За да може при рендирането да се приложат и характеристиките на камерата и избраната проекция, е нужно да бъде модифициран методът **render** в **Renderer / OpenGLRenderer**. Тук като параметър вече ще се добави и обект Camera, като към буфера ще се подават изчислените матрици на камерата и проекцията.

```
public void render(Scene scene, Camera camera) {
  shader.use();
  shader.setUniform("viewMatrix", camera.getViewMatrix());
  shader.setUniform("projectionMatrix", camera.getProjectionMatrix());

  for (SceneObject sceneObject : scene.getSceneObjects()) {
    Transform transform = sceneObject.getTransform();
    shader.setUniform("modelMatrix", transform.getModelMatrix());
    sceneObject.render(shader);
  }
}
```

При преизчисляването на позициите на върховете във вертексния шейдър вече отношение ще вземат и тези две матрици. Обърнете внимание на реда на подаването на трите трансформационни матрици.

```
#version 460 core

layout (location = 0) in vec3 aPos;
layout (location = 1) in vec3 aColor;
uniform mat4 modelMatrix;
uniform mat4 viewMatrix;
uniform mat4 projectionMatrix;

out vec3 vertexColor;

void main() {
  gl_Position = projectionMatrix * viewMatrix * modelMatrix * vec4(aPos, 1.0);
  vertexColor = aColor;
}
```

В класа App добавете поле за камера. Актуализирайте извикването на изменения метод render().

```
private Camera camera;
```

Самото добавяне на избраните параметри на камерата и проекцията се извършва в метода му за инициализация:

```
@Override
protected void init() {
  setWindow(new Window(width, height, title));

  scene = SceneBuilder.build();

  renderer = new OpenGLRenderer();
```

```
  renderer.init();

  camera = new PerspectiveCamera(
        (float) Math.toRadians(60),
        0.1f,
        100f,
        getWindow().getWidth(),
        getWindow().getHeight()
  );

 /*  camera = new OrthographicCamera(-3f, 3f,
        -3f, 3f,
        0.1f, 100f);*/

  //camera.setPosition(new Vector3f(2f, -2f, 3f));
 // camera.setTarget(new Vector3f(0.3f,0.4f,0f));
  //camera.setUp(new Vector3f(0f,1f,0f));
}
```

Тествайте добавения код.

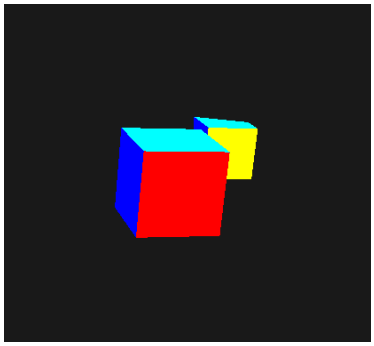
- Променете ъгъла на видимост на проекцията.
- Изменете позицията на камерата.
- Изменете точката, към която гледа камерата.

Задача за самостоятелна работа

Задача 1

В допълнение на задачата по-горе, изпълнете следните стъпки:

1. Премахнете анимираната ротация и завъртете куба по x и y с 18 градуса (или друг подходящ ъгъл, с който да се наблюдава триизмерността);
2. Мащабирайте го с коефициент 0.5 (или друг избран от Вас коефициент);
3. Добавете втори куб със същите размери, но с транслирани координати. Нека да застане отчасти зад първия куб.



4. Променете параметрите на камерата и наблюдавайте сцените.
5. Заменете перспективната проекция с ортографска.

Задача 2

Даден е връх с координати (-0.5, 0.5, 0.5). Върху него са приложени следните трансформации: ротация около оста Z с ъгъл 90 градуса; транслация с вектор (0.1, 0.1, 0.1) и скалиране с коефициент 0.9 за всички координати (съобразете подходящият ред на прилагане). Наблюдаван е от позиция (1, 0, 3), като камерата гледа към точка с координати (0.2, 0.2, 0.2) и е ориентирана нагоре по отношение на оста Y. Приложена е перспективна проекция с ъгъл на видимост 90 градуса. Отношението между страните е 1:1. Отстояния до близката и далечната равнина – 0.1 и 100.

Намерете:

1) Моделната матрица, с помощта на която ще се приложи трансформацията в световното пространство;
2) Матрицата на изгледа;
3) Проекционната матрица;
4) Координатите на върха в световното пространство;
5) Координатите на върха в координатната система на камерата;
6) Координатите на върха след прилагане на проекцията.
7) Нормализираните координати след прилагане на перспективно деление с хомогенните координати.
8) Да се намерят координатите на върха в координатната система на устройството (screen coordinates), като се използват следните параметри на екрана: ширина $width = 800$ пиксела и височина $height = 600$ пиксела.
9) Вместо перспективна проекция, приложете ортографска проекция със следните параметри:

$$left = -2, right = 2, bottom = -2, top = 2, near = 0.1, far = 100$$